Cambridge (CIE) IGCSE Computer Science

Your notes

Standard Methods of a Solution

Contents

Linear Search & Bubble Sort

Your notes

Linear Search

What is a searching algorithm?

- Searching algorithms are **precise step-by-step instructions** that a computer can follow to **efficiently locate specific data** in massive datasets

What is a linear search?

- A linear search **starts with the first value** in a dataset and **checks every value one at a time** until all values have been checked
- A linear search can be performed **even if the values are not in order**

How do you perform a linear search?

Step	Instruction
1	Check the first value
2	**IF** it is the value you are looking for ▪ STOP!
3	**ELSE** move to the next value and check
4	**REPEAT UNTIL** you have checked all values and not found the value you are looking for

Examiner Tips and Tricks

You will not be asked to perform a linear search on a dataset in the exam, you will be expected to understand how to do it and know the advantages and disadvantages of performing it

A linear search in Pseudocode

```
# Identify the dataset, target value, and set the initial flag
DECLARE data : ARRAY[1:5] OF INTEGER
DECLARE target : INTEGER
DECLARE found : BOOLEAN

data ← [5, 2, 8, 1, 9]
target ← 11
found ← FALSE

# Loop through each element in the data
FOR index ← 1 TO 5 DO
   IF data[index] = target THEN
      # If found, output message
      found ← TRUE
      OUTPUT "Target found"
      BREAK
   ENDIF
NEXT index

# If the target is not found, output a message
IF found = FALSE THEN
   OUTPUT "Target not found"
ENDIF
```

A linear search in Python code

```
# Identify the dataset to search, the target value and set the initial flag
data = [5, 2, 8, 1, 9]
target = 11
found = False

# Loop through each element in the data
for index in range(0, len(data)):  # loop to go through all elements
   # Check if the current element matches the target
   if data[index] == target:
      # If found, output message
      found = True
      print("Target found")
      break  # Exit the loop if the target is found

# If the target is not found, output a message
if not found:
   print("Target not found")
```

Bubble Sort


Your notes

What is a sorting algorithm?

- Sorting algorithms are **precise step-by-step instructions** that a computer can follow to **efficiently sort data** in massive datasets

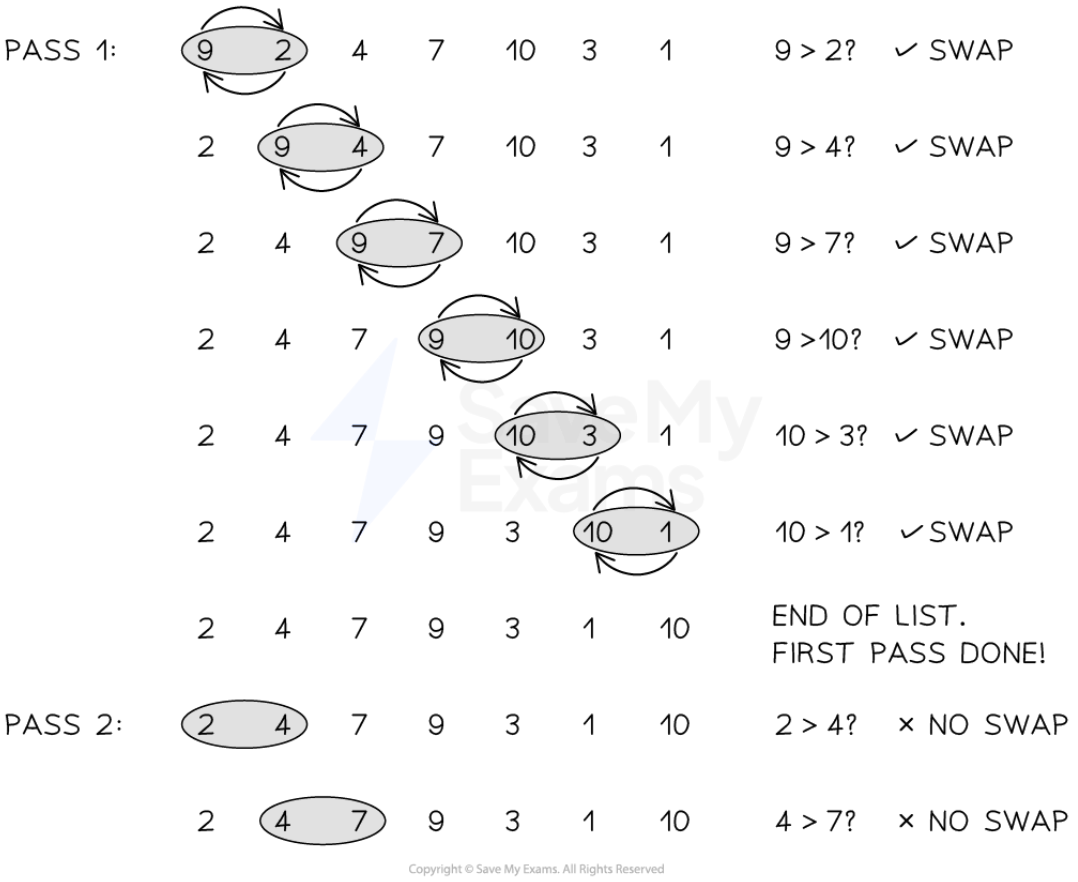
What is a bubble sort?

- A bubble sort is a simple sorting algorithm that starts at the beginning of a dataset and **checks values** in **'pairs'** and **swaps them** if they are **not in the correct order**
- One full run of comparisons from beginning to end is called a '**pass**', a bubble sort may require multiple '**passes**' to sort the dataset
- The algorithm is finished when there are **no more swaps to make**

How do you perform a bubble sort?

Step	Instruction
1	Compare the first two values in the dataset
2	**IF** they are in the wrong order... ▪ Swap them
3	Compare the next two values
4	**REPEAT** step 2 & 3 until you reach the end of the dataset (**pass 1**)
5	**IF** you have made any swaps... ▪ **REPEAT** from the start (pass 2,3,4...)
6	**ELSE** you have not made any swaps... ▪ STOP! the list is in the correct order


Your notes



Your notes

Example

- Perform a bubble sort on the following dataset

5	2	4	1	6	3

Step	Instruction
1	Compare the first two values in the dataset 5 | 2 | 4 | 1 | 6 | 3
2	**IF** they are in the wrong order...


Your notes

- Swap them

2	5	4	1	6	3

3 Compare the next two values

2	5	4	1	6	3

4 **REPEAT** step 2 & 3 until you reach the end of the dataset

- 5 & 4 SWAP!

2	4	5	1	6	3

- 5 & 1 SWAP!

2	4	1	5	6	3

- 5 & 6 NO SWAP!

2	4	1	5	6	3

- 6 & 3 SWAP!

2	4	1	5	3	6

- End of pass 1

5 **IF** you have made any swaps...

- **REPEAT** from the start
- End of pass 2 (swaps made)

2	1	4	3	5	6

- End of pass 3 (swaps made)

	<table><tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td></tr></table> ▪ End of pass 4 (no swaps) <table><tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td></tr></table>
6	**ELSE** you have not made any swaps... ▪ STOP! the list is in the correct order

Your notes

Examiner Tips and Tricks

In the exam you do not have to show every swap that takes place in a bubble sort. You can show the outcome of a bubble sort at the end of each pass. If you have the outcome of each pass correct then a bubble sort has been implemented correctly and all marks will be given!

A bubble sort in Pseudocode

```
# Unsorted dataset
DECLARE nums : ARRAY[1:11] OF INTEGER
nums ← [66, 7, 69, 50, 42, 80, 71, 321, 67, 8, 39]

# Count the length of the dataset
DECLARE numlength : INTEGER
numlength ← 11

# Set a flag to initiate the loop
DECLARE swaps : BOOLEAN
swaps ← TRUE

WHILE swaps DO  # While any swap is made, continue
  swaps ← FALSE
  # Loop through the dataset
  FOR y ← 1 TO numlength - 1 DO
    IF nums[y] > nums[y + 1] THEN  # If the first number is bigger
      # Swap the numbers
      DECLARE temp : INTEGER
      temp ← nums[y]
      nums[y] ← nums[y + 1]
      nums[y + 1] ← temp
      swaps ← TRUE  # Mark that a swap was made
```

```
    ENDIF
  NEXT y
  # Each iteration confirms that the last element is in place
  numlength ← numlength - 1
ENDWHILE

# Print the sorted list
OUTPUT nums
```

A bubble sort in Python code

```
# Unsorted dataset
nums = [66, 7, 69, 50, 42, 80, 71, 321, 67, 8, 39]

# Count the length of the dataset
numlength = len(nums)

# Set a flag to initiate the loop
swaps = True

while swaps:  # While any swap is made, continue
  swaps = False
  # Loop through the dataset
  for y in range(numlength - 1):  # Compare adjacent elements
    if nums[y] > nums[y + 1]:  # If the first number is bigger
      # Swap the numbers using a temporary variable
      nums[y], nums[y + 1] = nums[y + 1], nums[y]
      swaps = True  # Mark that a swap was made

  # Each iteration confirms that the last element is in place
  numlength -= 1

# Print the sorted list
print(nums)
```

Your notes

Worked Example

A program uses a file to store a list of words.

A sample of this data is shown

Milk	Eggs	Bananas	Cheese	Potatoes	Grapes

Show the stages of a bubble sort when applied to data shown **[2]**

How to answer this question

- We need to sort the values in to **alphabetical order** from A-Z
- You **CAN** use the first letter of each word to simplify the process

Answer

- E, B, C, M, G, P (pass 1)
- B, C, E, G, M, P (pass 2)

Your notes

Other Standard Methods of a Solution

Your notes

Totalling & Counting

What is totalling?

- Totalling is keeping **a running total** of values entered into the algorithm
- An example may be totalling a receipt for purchases made at a shop
- In the example below, the total starts at 0 and adds up the user inputted value for each item in the list

Pseudocode

```
Total ← 0
FOR Count ← 1 TO ReceiptLength
   INPUT ItemValue
   Total ← Total + itemValue
NEXT Count
OUTPUT Total
```

What is counting?

- Counting is when a count **is incremented or decremented by a fixed value**, usually 1, each time it iterates
- Counting **keeps track of the number of times an action has been performed**
- Many algorithms use counting, including the linear search to track which element is currently being considered
- In the example below, the count is **incremented** and each pass number is output until fifty outputs have been produced

Pseudocode

```
Count ← 0
DO
   OUTPUT "Pass number", Count
   Count ← Count + 1
UNTIL Count >= 50
```

- In the example below, the count is **decremented** from fifty until the count reaches zero. An output is produced for each pass

Pseudocode

```
Count ← 50
DO
   OUTPUT “Pass number”, Count
   Count ← Count - 1
UNTIL Count <= 0
```

Your notes

Maximum, Minimum & Average

- Finding the largest (**max**), smallest (**min**) and average (**mean**) values in a list are frequently used method in algorithms
- Examples could include:
  - Calculating the maximum and minimum student grades or scores in a game
  - Calculating the average grade of students in a class test
- In the example below, in a list of student test scores, the highest, lowest and average scores are calculated and displayed to the screen

Pseudocode

```
Total ← 0
Scores ← [25, 11, 84, 91, 27]

Highest ← max(Scores)
Lowest ← min(Scores)

# Loop through the scores in the list (indexing starts from 0)
FOR Count ← 0 TO LENGTH(Scores) - 1
   # Add score to total
   Total ← Total + Scores[Count]
NEXT Count

# Calculate average
Average ← Total / LENGTH(Scores)

OUTPUT "The highest score is:", Highest
OUTPUT "The lowest score is:", Lowest
OUTPUT "The average score is:", Average
```